

Topic 1: Introduction to Programming and C++

What is Programming?

A program is a set of instructions that tells a computer what to do. Computers only understand binary (0s and 1s), so programming languages like C++ act as a bridge between human thinking and machine execution. Every app, game, or website you use is the result of a program written by a programmer.

Why C++?

C++ is one of the most powerful and widely-used programming languages in the world. It is used in game development (Unreal Engine), operating systems, embedded systems, competitive programming, and high-performance software. Learning C++ builds a strong foundation because it teaches you how memory works, how the CPU executes instructions, and how to think like a computer.

Compiled vs Interpreted Languages

C++ is a compiled language. You write source code (.cpp file), the compiler translates it into machine code (.exe or binary), and then the CPU runs it directly. This makes C++ programs extremely fast. Interpreted languages like Python run line by line through an interpreter, which is slower but more flexible.

Setting Up the Environment

Install VS Code with the C/C++ extension and MinGW compiler (Windows), or use Code::Blocks which comes bundled with a compiler

The compiler converts your .cpp file into an executable

The linker connects your code with standard library functions

The build process: write code → compile → link → run

Structure of a C++ Program

Every C++ program has the same basic skeleton:

`#include <iostream>` — tells the compiler to include the input/output library

`using namespace std;` — lets you use cout and cin without the std:: prefix

`int main()` — the entry point, execution always starts here

`return 0;` — signals the program ended successfully

Hello World Explained Line by Line

```
```cpp
```

```
#include <iostream>

using namespace std;

int main() {
 cout << "Hello, World!" << endl;
 return 0;
}
...

```

`cout` sends output to the screen

`<<` is the insertion operator, it pushes data into cout

`endl` ends the line and flushes the output buffer

## Types of Errors

Syntax error — you broke a grammar rule (missing semicolon, mismatched brackets). The compiler catches this before running.

Linker error — the compiler found your code but cannot find a function definition it needs.

Runtime error — the program compiled fine but crashes while running (e.g., dividing by zero).

Logic error — the program runs but gives the wrong answer. The hardest to find.

## Comments

Comments are notes in your code ignored by the compiler. Use them to explain what your code does.

```
```cpp

```

```
// This is a single-line comment

```

```
/* This is a

```

```
multi-line comment */

```

```
...

```

The Standard Library

The `#include` directive imports pre-written code. `<iostream>` gives you `cout` and `cin`.
`<string>` gives you the `string` type. `<cmath>` gives you math functions like `sqrt()` and `pow()`.
You will use these throughout the course.
